

FP RISC and QOS Reliability Design

Preliminary contracts and explicit intermediate states

Draft 0.1 | 5 October 2026

This document proposes a reliability framework for FP-RISC and QOS. Each subsystem should define what must never happen, what must eventually happen, what resources must remain bounded, and what each intermediate state means. Larger operations should be compositions of small protocols with explicit ownership, failure and cleanup rules.

The draft develops the preceding design discussion. It is not a repository audit or a claim that these guarantees are implemented. State names are illustrative; implementation mappings, numeric limits and evidence remain to be supplied.

The three kinds of contract

Safety A forbidden event never occurs: no stale completion is accepted, no freed actor executes, and no unauthorized buffer is accessed.

Liveness An enabled obligation eventually resolves under stated assumptions: an accepted request completes, fails, or is abandoned through a defined recovery path.

Boundedness Resources and work have explicit limits: queue depth, memory, retained results, retries, execution slices and recovery attempts.

The governing design rule

Every state in which scheduling, failure, cancellation or ownership can affect correctness must have explicit semantics. A name alone is insufficient: the state must identify its owner, held resources, enabled events and progress obligation.

Enumeration belongs at meaningful protocol boundaries. An uninterrupted local calculation does not need a public state for every instruction. Public APIs such as `readFileString` may remain convenient wrappers; their reliability model expands into smaller steps.

How to use this draft

Start with actor cleanup and one Files-to-block request path. Map actual code to the proposed states, write missing contracts, and attach failure tests. Expand coverage after those two paths have a complete model.

Universal obligations

ID	Proposed obligation
U1	Ownership and borrowing are explicit. Each resource has a defined lifetime owner; shared access has a synchronization rule.
U2	Within a defined request identity and service epoch, at most one terminal outcome becomes authoritative.
U3	Admission is observable. Rejected work creates no service obligation; accepted work creates a tracked resolution obligation.
U4	Stale messages, handles, interrupts and completions cannot refer to a reused object or service incarnation.
U5	Caller resolution and physical cleanup are tracked separately. Timeout never authorizes unsafe buffer reuse.
U6	Every wait names its dependency, wakeup event, deadline or other progress assumption, and failure escape.
U7	Memory, queueing, retries and recovery have explicit budgets and exhaustion behavior.
U8	Partial initialization and shutdown account for every acquired resource and in-flight operation.

Define the scope of eventual resolution

Liveness needs assumptions: fair scheduling, functioning timers, a surviving recovery authority and bounded dependencies where required. A dead device can lead to a failure result; permanent loss of every participant cannot guarantee result delivery. Explicitly distinguish local termination from delivery to a live caller.

Define exactly what has happened

Send means the transport accepted a message, unless its API specifies otherwise. Acceptance means the service admitted the operation. ReplySent means the reply transport accepted a result. ReplyReceived means the client consumed the correlated result. None of these events automatically implies durable storage.

At-most-once terminal publication does not prove exactly-once execution across crashes. Retrying a write may repeat a side effect. Durable deduplication, an idempotent operation, or an explicit outcome-unknown result is needed when effects cannot be reconstructed.

Failures that exceed containment

Specify which faults are recoverable and which require fail-stop behavior or reboot. A supervisor cannot safely recover arbitrary kernel memory corruption merely by restarting an actor.

Enumerating intermediate states

Model each participant locally. A client waiting for a read and a server executing it occupy different states. A diagnostic snapshot may combine them, but they do not form one automatically synchronized global state.

State family	Meaning and obligation
Created	Local request exists; no remote work is assumed.
Sending or Sent	Distinguish a blocked send from a transport-accepted message. Admission may still be unknown.
Accepted or Queued	Service owns a resolution obligation; queued resources count against admission limits.
Running or WaitingForX	Identify the active resource or dependency, such as BlockCompletion or IRQ.
ReplyReady or ReplySent	Specify who retains the result and how delivery failure or caller death releases it.
Cancelling or Quiescing	Cancellation is requested; work or hardware access may still be active.
Terminal or Reclaiming	Outcome is fixed; cleanup may remain. Reclaimed means all required lifetime obligations ended.

Required state record

For every meaningful state record: identifier and epoch; participant and owner; resources and borrows; legal incoming events; guarded outgoing transitions; publication or synchronization point; dependency and deadline; budget; cancellation and death behavior; diagnostic fields; and a test that exercises exit.

Transition rules

Express each transition as current state + event + guard → next state + effects. In concurrent code identify the lock, compare-and-swap or message serialization that makes the authoritative decision. No transition may publish two outcomes or free memory still reachable by another participant.

Unexpected events need a policy: reject, safely discard with resource release, or escalate an invariant violation. Late replies require disposal even when their values are ignored.

Observability

Trace request identity, epoch, old and new state, dependency, deadline and ownership changes. Use bounded traces and counters. A watchdog should report the state and blocked dependency, not just “file operation stalled”.

Worked example of a file read

Assume a correlated request identity made from client identity, sequence number and service epoch. Its buffer transfer rule must be specified by the IPC contract. Names below represent protocol events rather than proposed built-in operations.

Participant	Normal state sequence
Client	Created → Sending → FileReadReqSent → WaitingForFileReadResult → FileReadResultReceived(Result Bytes) → OutcomeRecorded
Files	FileReadReqReceived → Accepted → WaitingForBlockResult → ReplyReady → FileReadResultSent → Reclaiming → Reclaimed
Block driver	Queued → Submitted → WaitingForDeviceCompletion → CompletionValidated → OwnershipReturned

Events that change the normal path

Admission failure A full queue or allocation failure produces an explicit rejection. A send success alone does not establish service admission.

Caller death Files cancels or safely finishes admitted work and disposes of an undeliverable result. Device-owned memory stays alive until quiescence is established.

Service death A surviving authority resolves or invalidates outstanding requests under the restart contract. Old replies cannot satisfy requests in the new epoch.

Completion racing cancellation A serialized decision fixes the caller outcome. Cancellation may be best-effort; its acknowledgment must state whether it merely accepted cancellation or proves execution stopped.

Timeout The caller may enter TimedOut while the service is still Quiescing. Preserve service-side ownership until completion, reset or another supported mechanism proves hardware no longer accesses the buffer.

Two distinct completion obligations

Normal protocol completion is the client receiving Result Bytes. If it times out or dies, that receive may never occur. Record a local terminal outcome separately from service cleanup. Reclaimed is a resource state, not evidence that a reply reached the client.

Read results need defined semantics

Specify whether a read can be partial, what end-of-file means, and which consistency guarantees apply during concurrent writes. A reply identifies both the request and its result; protocol correlation must not depend on arrival order.

Composing small protocols

Define high-level operations through explicit control-flow composition. A protocol transaction here means a conversation with a lifetime and outcome; it does not imply database atomicity, rollback, isolation or durability.

Operator	Proposed meaning
A ; B	Run B after successful A. Pass its value and ownership forward; propagate failure according to an explicit rule.
choice A B	Select a guarded alternative. State who chooses and whether the unselected branch acquired resources.
A B	Run both; define joining, sibling cancellation and cleanup when either fails.
timeout A t	Bound caller waiting using a defined clock. Continue tracking any work that outlives the caller result.
retry A policy	Use attempt and elapsed-time budgets; specify request identity and duplicate-effect handling.
bracket acquire use release	Release after successful acquisition on every supported exit from use; define cleanup failure handling.

A file to string wrapper

Conceptually: `bracket(open(path), handle → readAll(handle), close) ; decodeUtf8(bytes)`. A read expands into `construct request ; send request ; receive correlated result ; match result`. `readAll` repeats reads with a maximum byte budget and a defined EOF condition.

If read fails, close is still attempted. If close also fails, preserve the primary error and attach the cleanup error. Actor death needs a service or runtime ownership rule because local finally code may never run. Successful UTF-8 decoding must not hide a failed close.

Conditions for compositional liveness

Each step must resolve under its assumptions, success must enable the next step, and resource ownership must connect across the boundary. Loops require a decreasing measure, finite budget or explicitly persistent contract. Parallel branches must not hold resources that the other branch needs to complete.

Deadlock review

Build a wait-for graph over actors, locks, queue capacity and device dependencies. A cycle matters when every edge blocks and no alternative transition can break it. Include backpressure: a full reply queue can deadlock a caller waiting for a reply while holding a resource the service needs. Use a consistent lock order or remove blocking dependencies where possible.

FP RISC compiler and language contracts

Parsing typing and module resolution

Safety Malformed input never becomes accepted executable code. Names, nominal types and module identities preserve their specified meaning.

Liveness Compilation returns a result or a defined resource-limit error for supported inputs. Detect dependency cycles. Unrestricted compile-time evaluation cannot promise universal termination.

Boundedness Bound nesting, diagnostics, import expansion and inference work. Report limits rather than exhausting host memory.

Intermediate states SourceLoaded → Parsed → NamesResolved → Typed; module lookup may be WaitingForDependency or CycleRejected.

Lowering optimization and code generation

Safety Successful compilation preserves defined source behavior. Check pattern coverage, value layout, register preservation, ARC placement, calling conventions and exception or error paths.

Liveness Passes finish within their work budget. Recursive transformations need a measure or explicit cutoff.

Boundedness Bound IR growth, specialization, inlining and generated code. Budget exhaustion has a documented compilation outcome.

Intermediate states TypedIR → LoweredIR → OptimizedIR → CodeEmitted → ArtifactValidated.

Language semantics ABI and artifacts

Safety Pin integer widths and overflow, floating-point behavior, evaluation order, measures, record identity and ownership rules. Reject incompatible modules and ABI versions before execution.

Liveness Compatible artifacts load or fail with a reason. Differential checks must distinguish allowed platform variation from semantic disagreement.

Boundedness Bound artifact sizes and validation effort. Cache identity includes compiler, configuration, dependencies and ABI.

Intermediate states ArtifactRead → CompatibilityChecked → Linked → Published; incomplete output never masquerades as a successful artifact.

Useful evidence

Compare reference semantics and optimized execution; compare backends for the supported common language subset. Generate small well-typed programs, retain regressions for discovered mismatches, and test malformed source under budgets. Compare observable results, errors and permitted effects rather than unspecified instruction timing.

Runtime memory actors and IPC

Memory and reference counting

Safety No use-after-free, double release, invalid borrow or size overflow. Shared references obey synchronization. Specify whether reference cycles are forbidden, collected or explicitly broken.

Liveness Unreachable reclaimable resources are eventually released, including cancellation paths. OOM yields a defined failure or fail-stop outcome.

Boundedness Bound actor pools, stacks, heaps and retained references; test steady resource use across repeated create and destroy cycles.

Intermediate states Allocated → Initialized → SharedOrBorrowed → Retired → Reclaimed.

Scheduler and actor lifecycle

Safety An actor executes on only its authorized scheduler owner; state transitions and wakeups cannot expose freed state.

Liveness Runnable actors eventually run under fairness assumptions. Killed blocked actors become eligible for owner-side reaping. Specify cooperative versus preemptive scheduling.

Boundedness Bound runnable queues and execution slices. Cooperative actors require yielding rules or a watchdog escalation policy.

Intermediate states Created → Runnable → Running → WaitingForX → Runnable; termination follows KillRequested → Quiescing → Reaping → Dead.

IPC channels and capabilities

Safety Messages remain correlated and authorized. Define copying or transfer, revocation behavior, channel closure and generation reuse.

Liveness Accepted sends deliver or obtain a defined failure under channel assumptions. Closure wakes blocked senders and receivers.

Boundedness Bound mailbox size, retained payload bytes and outstanding RPCs. Specify rejection or backpressure without cyclic blocking.

Intermediate states SendPrepared → WaitingForCapacity → Enqueued → Received; channel lifetime is Open → Closing → Drained → Closed.

Timers and interrupts

Safety Cancelled or stale timers and IRQ bindings cannot wake a recycled actor. Interrupt context uses only permitted nonblocking mechanisms.

Liveness Expired timers and pending interrupts become visible under masking and dispatch assumptions; a lost IRQ has a recovery path.

Boundedness Bound timer entries, ISR work and deferred IRQ queues. Interrupt storms have a masking or rate policy.

Intermediate states TimerArmed → Expired → WakeQueued → Delivered; IRQPending → Claimed → Deferred → Handled → Acknowledged.

Storage and device contracts

Files handles and namespaces

Safety No wrong-file access, stale-handle reuse or unauthorized namespace crossing. Specify read/write consistency, partial results and close semantics.

Liveness Accepted operations resolve or are invalidated by an explicit service-death rule. Close drains or cancels relevant work.

Boundedness Bound handles, file sizes where required, buffered bytes, request queues and per-client outstanding operations.

Intermediate states OpenRequested → HandleGranted → ReadOrWriteWaiting → ResultReceived; close follows Closing → Draining → Closed.

Block I O and DMA

Safety Validate correlation and generation before accepting completion. DMA buffers remain valid until hardware ownership returns. Apply the platform memory-ordering and cache-coherency rules.

Liveness Accepted requests complete, fail or enter bounded recovery. Timeout does not prove a device has stopped DMA.

Boundedness Bound descriptors, pinned memory, queue depth, retries and resets. If quiescence cannot be proven, escalate rather than reuse unsafe memory.

Intermediate states Queued → Submitted → DeviceOwnsBuffer → CompletionValidated → BufferReturned; failure may enter Resetting → Quiesced.

Persistent log and crash recovery

Safety Define ordering, checksums, torn-write handling and the durability point. An acknowledgment means durable only after the required device flush or barrier succeeds.

Liveness Recovery restores a valid prefix or reports an unrecoverable condition. Full media and failed flushes have defined outcomes.

Boundedness Bound log growth, replay work and checkpoint resources; define retention and space-exhaustion policies.

Intermediate states RecordPrepared → Appended → FlushPending → Durable → Checkpointed; boot may enter Scanning → Validated → Recovered.

Driver binding and reset

Safety Driver death, removal or reset cannot leave old requests targeting reused device state. Define authority to stop hardware and invalidate bindings.

Liveness Initialization either becomes Ready or unwinds. Reset reaches Ready, DeviceFailed or system escalation within a recovery budget.

Boundedness Bound initialization retries, reset attempts and interrupt storms. Reserve resources required for recovery.

Intermediate states Discovered → Binding → Initializing → Ready → Draining → ResettingOrRemoving → Unbound.

Network loading and user facing services

Network and connections

Safety Validate lengths, state and checksums before mutation. Stale connection identifiers do not affect new connections; define delivery and retransmission semantics per layer.

Liveness Connection attempts and sends progress or timeout under network assumptions. Peer loss and service restart resolve local waiters.

Boundedness Bound connections, packet queues, reassembly, retransmissions and per-peer state. Pressure has an explicit drop or admission policy.

Intermediate states ConnectCreated → WaitingForNeighbor → WaitingForHandshake → Established → Closing → Closed; sends include Queued and WaitingForAck.

Loader packages and processes

Safety Validate image format, bounds, ABI and capabilities before publishing an executable process. Failed loads leave no executable partial state.

Liveness Load succeeds or unwinds; dependency cycles and unavailable services return errors. Exit resolves owned resources.

Boundedness Bound image size, dependency depth, relocation work, processes and temporary load memory.

Intermediate states ImageRead → Validated → ResourcesReserved → Linked → Initialized → Published; failure enters Unwinding.

Graphics audio and input

Safety Frame and audio buffers remain valid while consumed. Input events target a live authorized recipient. Reconfiguration cannot reuse an active buffer.

Liveness Presentation, playback and device changes complete or report loss. Missing consumers cannot permanently block a service.

Boundedness Bound frames, audio buffers and event queues; specify frame drops, audio underrun behavior and input overflow policy.

Intermediate states BufferPrepared → Queued → DeviceConsuming → Released; input follows Captured → Routed → ConsumedOrDropped.

TTY shell and logging

Safety Untrusted text cannot corrupt service state. Logging is safe in failure paths and IRQ context; emergency output avoids locks held by the failing code.

Liveness TTY work resolves or is interrupted. Logs drain under sink assumptions; a failed sink cannot freeze unrelated services.

Boundedness Bound line lengths, scrollbar, log bytes and formatting work. Count truncation or dropped records explicitly.

Intermediate states TTYReadWaiting → InputReady → ReadResolved; LogPrepared → Queued → SinkWaiting → WrittenOrDropped.

Supervision shutdown and verification

Supervision startup and shutdown

Safety Restarts change epochs and fence obsolete owners. Partial startup unwinds acquired resources. Shutdown prevents new admission before draining and release.

Liveness Failed services restart or become definitively unavailable within budget. Shutdown finishes or takes a defined escalation path.

Boundedness Bound restart frequency, startup work, shutdown deadlines and retained orphan requests. Prevent restart storms.

Intermediate states Starting → DependenciesReady → Serving → AdmissionClosed → Draining → Quiescing → Released → Stopped; failures enter RestartPending or Unavailable.

Derive tests from state transitions

For each state inject allocation failure, delayed or missing completion, duplicate or stale message, cancellation, caller death, service death and shutdown where applicable. Exercise both event orders at race boundaries, not just the expected sequence. Test device-reset and power-loss boundaries separately from actor failure.

Each test needs an oracle: forbidden outcomes, expected terminal result, resource ownership, maximum resolution time and final resource count. For a timed-out DMA request, specifically verify that caller resolution precedes reuse only when quiescence is proven.

A first implementation pass

1 Inventory actual subsystems and map code to states. Mark each guarantee as Proposed, Implemented, Tested or Known gap; attach code and test references to every promotion.

2 Complete the actor kill and reaping path, then one Files read path through the block driver. Include cancellation, reset, restart and undeliverable reply cleanup.

3 Write numeric budgets and clock assumptions. Add transition traces, invariant assertions and deterministic fault-injection hooks.

4 Run repeated lifecycle tests, multi-hart races, long-running resource checks and cross-backend semantic comparisons. Use small finite state models for difficult cancellation and ownership races.

Questions to settle next

What event constitutes admission? Who owns each reply buffer? Which cancellation acknowledgments prove quiescence? How are epochs persisted or invalidated? Which writes can return outcome unknown? What fairness is promised? Which faults require reboot? What are the limits for each queue and recovery loop?

This draft becomes a reliability specification when those choices have concrete answers and each contract has evidence. Explicit states make missing answers visible; they do not by themselves prove correctness.